

PROJET KEPLER — CIR1

Recapitulatif complet du noyau de calcul en C

Projet	Kepler — Simulation du systeme solaire
Ecole	ISEN Brest / Yncrea Ouest — CIR1 2026
Rendu	Mardi 16/06/2026 a 12h00
Langage C	gcc -Wall -Wextra, Makefile avec wildcard
Methode principale	RK2 symplectique + substeps adaptatifs
Corps simules	Soleil + 8 planetes + 11 satellites + Halley
Pas de temps	dt=1800s (planetes), dt_i=300s (satellites rapides)
Duree simulee	5 ans (planetes), 10 ans (Halley)

1. Introduction et contexte

Ce document presente de maniere exhaustive le noyau de calcul en C du projet Kepler. Il couvre l'architecture choisie, les structures de donnees, les methodes numeriques implementees, les problemes rencontres et leur resolution, ainsi que les comparaisons avec les pratiques professionnelles en mecanique celeste.

Le sujet du TP demandait a minima : calculer la trajectoire de la Terre autour du Soleil par la methode d'Euler, exporter les donnees en JSON, et afficher les trajectoires graphiquement. Notre implementation depasse largement ce minimum en implementant quatre methodes d'integration, un systeme solaire complet avec satellites, des orbites elliptiques reelles, un barycentre dynamique, et une technique d'integration multi-echelle.

2. Architecture du noyau C

2.1 Organisation des fichiers

Le sujet recommandait de separer les fichiers .h, .c et .o dans des dossiers distincts. Nous avons adopte la structure suivante :

Fichier	Role	Contenu principal
vector.c/.h	Brique de base	Vector3, add, sub, scale, norm, test
point.c/.h	Instant de trajectoire	struct Point : position, velocity, time
trajectory.c/.h	Liste de points	malloc, traj_add, traj_free, traj_print
body.c/.h	Corps celeste generique	struct Body : name, mass, trajectory
physics.c/.h	Moteur physique	4 methodes + system_simulate_adaptive
orbital_init.c/.h	Conditions initiales	Perihelie, vis-viva, orbite satellite
energy.c/.h	Conservation energie	Ec, Ep, E totale, drift
solar_system.c/.h	Orchestration	Simulation complete 20 corps
json_export.c/.h	Export donnees	Format impose, sous-echantillonnage
constants.h	Constantes physiques	Masses, perihelies, excentricites, inclinaisons

2.2 Choix de l'architecture Body generique

Le sujet proposait des structures Planet et Satellite separees. Nous avons choisi une structure Body unique et generique qui fonctionne pour n'importe quel corps celeste (planete, satellite, comete, vaisseau). Ce choix simplifie le code car une meme fonction body_simulate fonctionne pour tous les types de corps, et l'ajout d'un nouveau corps ne necessite aucune modification des fonctions existantes.

Choix technique

Body generique au lieu de Planet + Satellite separees. Avantage : extensibilite. Un asteroide, une comete, un vaisseau sont tous des Body avec les memes fonctions.

3. Physique implementee

3.1 Loi de la gravitation — ce que demandait le sujet

Le sujet donnait la formule de la loi universelle de Newton (equation 5) sous forme vectorielle :

$$F(A/B) = -G * m_A * m_B / ||r||^3 * r$$
$$a = -(G * M_{\text{attracteur}} / ||r||^3) * r$$

Notre implementation (physics.c) generalise cette formule a n attracteurs par principe de superposition :

```
Vector3 compute_acceleration_from(Vector3 pos_body, Vector3 pos_attractor, double
mass_attractor) {
    Vector3 r_vec = vec_sub(pos_body, pos_attractor);
    double r = vec_norm(r_vec);
    double factor = -(G * mass_attractor) / (r * r * r);
    return vec_scale(r_vec, factor);
}
```

La fonction total_acceleration somme les contributions de tous les attracteurs, implementant le principe de superposition. Chaque corps est attire par TOUS les autres corps du systeme simultanement.

3.2 Valeurs des constantes physiques

Le sujet donnait $G = 6.67408e-11 \text{ N.kg}^{-2}.\text{m}^2$. Toutes les autres constantes proviennent de sources officielles :

Corps	Masse (kg)	Perihelie (m)	Excentricite
Soleil	1.989e+30	—	—
Mercure	3.285e+23	4.600e+10	0.2056
Venus	4.867e+24	1.075e+11	0.0068
Terre	5.972e+24	1.471e+11	0.0167
Mars	6.390e+23	2.067e+11	0.0934
Jupiter	1.898e+27	7.405e+11	0.0489
Saturne	5.683e+26	1.353e+12	0.0565
Uranus	8.681e+25	2.742e+12	0.0457
Neptune	1.024e+26	4.445e+12	0.0113

Source : NASA Planetary Fact Sheets (<https://nssdc.gsfc.nasa.gov/planetary/factsheet/>). Les inclinaisons orbitales proviennent de la meme source (Mercure 7.005 deg, Venus 3.395 deg, etc.).

4. Methodes d'integration numerique

4.1 Ce que demandait le sujet

Le sujet demandait a minima la methode d'Euler (section 2.2.1) et proposait comme extensions la methode de Runge-Kutta d'ordre 2 (section 2.2.2) et la methode d'Euler asymetrique (section 2.2.3). Nous avons implemente les quatre methodes et ajoute une cinquieme : le RK2 symplectique.

4.2 Euler simple — methode minimale du sujet

Le sujet donnait les equations 11 et 13 :

```
// Ordre impose par le sujet :  
// 1. calcul des accelerations depuis les positions au temps tn  
// 2. calcul des positions au temps tn+1  
// 3. calcul des vitesses au temps tn+1
```

```
Vector3 acc      = total_acceleration(current.position, attractors, n);  
next.position    = vec_add(current.position, vec_scale(current.velocity, dt));  
next.velocity    = vec_add(current.velocity, vec_scale(acc, dt));
```

Probleme observe : derive d'energie de 15.87% apres 365 jours avec $dt=86400s$. La trajectoire diverge car l'acceleration utilisee est celle du debut du pas, pas la moyenne sur le pas. Le sujet lui-meme note que 'la trajectoire diverge rapidement'.

4.3 Euler asymetrique — extension du sujet

Le sujet (section 2.2.3) decrit cette methode comme 'plus simple a mettre en place que Runge-Kutta' et note que 'contrairement a la methode d'Euler, la divergence est faible'. La seule difference avec Euler simple est l'ordre des calculs :

```
// Position calculee EN PREMIER  
next.position    = vec_add(current.position, vec_scale(current.velocity, dt));  
// Acceleration depuis la NOUVELLE position (difference cle)  
Vector3 acc      = total_acceleration(next.position, attractors, n);  
// Vitesse depuis cette acceleration plus recente  
next.velocity    = vec_add(current.velocity, vec_scale(acc, dt));
```

Resultat obtenu : derive d'energie de seulement 0.0002% apres 365 jours — 80 000 fois meilleure qu'Euler simple. Cette methode est un integrateur symplectique d'ordre 1, aussi appele 'methode d'Euler symplectique' ou 'Euler-Cromer'. Elle conserve une quantite appelee energie symplectique, proche de l'energie reelle du systeme.

Pourquoi c'est meilleur

En calculant la position avant la vitesse, position et vitesse restent coherentes entre elles. Cette coherence est la definition d'un integrateur symplectique — il preserve la structure geometrique de l'espace des phases.

4.4 RK2 — extension du sujet

Le sujet decrit RK2 (equations 19-24) comme une methode qui subdivise l'intervalle de temps pour affiner la resolution. Notre implementation :

```
Vector3 k1_r = vec_scale(current.velocity, dt);  
Vector3 k1_v = vec_scale(total_acceleration(current.position, ...), dt);
```

```

Vector3 mid_pos = vec_add(current.position, vec_scale(k1_r, 0.5));
Vector3 mid_vel = vec_add(current.velocity, vec_scale(k1_v, 0.5));
Vector3 k2_r = vec_scale(mid_vel, dt);
Vector3 k2_v = vec_scale(total_acceleration(mid_pos, ...), dt);
next.position = vec_add(current.position, k2_r);
next.velocity = vec_add(current.velocity, k2_v);

```

Resultat : derive 0.0004% — bon mais legerement moins stable qu'Euler asymetrique sur le long terme car RK2 n'est pas symplectique. L'energie derive lentement sur des milliers d'annees.

4.5 RK2 symplectique — notre extension originale

Cette methode n'est pas dans le sujet. Elle combine la precision de RK2 (deux evaluations de derivee par pas) avec la stabilite de l'Euler asymetrique (logique position-avant-acceleration a chaque sous-pas). C'est un integrateur symplectique d'ordre 2.

```

// Demi-pas avec logique asymetrique
Vector3 half_pos = vec_add(current.position, vec_scale(current.velocity, dt*0.5));
Vector3 half_acc = total_acceleration(half_pos, attractors, n);
Vector3 half_vel = vec_add(current.velocity, vec_scale(half_acc, dt*0.5));
// Pas complet depuis le point milieu avec logique asymetrique
next.position = vec_add(half_pos, vec_scale(half_vel, dt*0.5));
Vector3 full_acc = total_acceleration(next.position, attractors, n);
next.velocity = vec_add(half_vel, vec_scale(full_acc, dt*0.5));

```

Resultat : derive 0.0000% — parfaite conservation de l'energie. C'est la methode utilisee pour toute la simulation finale.

Utilisation professionnelle

Le simulateur REBOUND (Tamayo et al., 2019, Journal of Open Source Software) utilise exactement cette famille d'integrateurs symplectiques pour simuler des systemes planetaires. L'integrateur IAS15 de REBOUND est une version d'ordre 15 de ce principe. La mission Cassini (NASA) a utilise des integrateurs symplectiques pour calculer les manoeuvres autour de Saturne.

4.6 Tableau comparatif des resultats

Methode	Calculs/pas	Symplectique	Derive energie	Erreur pos. 365j
Euler simple	1	Non	15.87%	~11%
Euler asymetrique	1	Oui	0.0002%	~1.1%
RK2	2	Non	0.0004%	~1.2%
RK2 symplectique	2	Oui	0.0000%	~1.1%

Ces resultats ont ete obtenus avec dt=86400s (1 jour), 365 pas, simulation Terre-Soleil uniquement.

5. Problemes rencontres et solutions

5.1 Phobos qui se detache de Mars

Probleme

Phobos orbite Mars en 7h30 (27 553 secondes). Avec $dt=1800s$, il y avait seulement 15 pas par orbite de Phobos — insuffisant pour maintenir la stabilite numerique. De plus, pendant chaque pas de 1800s, Mars se deplacait de 43 200 km (vitesse orbitale * dt), soit 4.6 fois le rayon orbital de Phobos (9 376 km). Phobos ne pouvait pas suivre Mars.

Tentatives echouees

- Simulation separee planetes / satellites : incorrecte car les corps n'interagissent pas entre eux
- simulate_planet_system avec interpolation de la planete : complexe et toujours instable
- Corps temporaires recrees a chaque substep : 1 million de malloc/free corrompait le heap

Solution finale

Integration multi-echelle dans system_simulate_adaptive : tous les corps sont dans une seule simulation. A chaque pas global de 1800s, les corps lents (planetes) avancent en 1 substep de 1800s, les corps rapides (Phobos, Deimos, Io, Europa, Triton, Proteus) avancent en 6 substeps de 300s. Chaque substep utilise les positions mises a jour des autres corps au meme substep.

```
// Principe : max_sub = 6 (le plus grand substep)
// dt_min = 1800 / 6 = 300s
// A chaque micro-pas de 300s :
//   Phobos avance de 300s (tous les micro-pas)
//   Mars   avance de 300s (tous les micro-pas)
//   Terre  avance de 900s (tous les 3 micro-pas)
//   Jupiter avance de 1800s (1 seul micro-pas sur 6)
```

Technique standard

Cette technique s'appelle 'integrateur multi-echelle' ou 'mixed variable symplectic integrator'. Elle est decrite dans Wisdom & Holman (1991), The Astronomical Journal, qui l'utilisent pour simuler le systeme solaire sur des milliards d'annees.

5.2 Segmentation fault lors de l'export JSON

Probleme

Le programme se terminait en segfault apres l'export JSON. La cause : le tableau to_export declarait 20 corps mais json_export_all_sampled etait appele avec $n=21$. La fonction lisait un pointeur hors du tableau.

Solution

Correction du comptage : 8 planetes + 11 satellites + 1 comete = 20 corps. Regle generale : toujours compter manuellement et verifier que n correspond exactement a la taille du tableau.

5.3 Trajectoires pleine pendant la simulation

Probleme

Le message 'Trajectoire pleine !' apparaissait des centaines de fois. Causes identifiees :

- Le Soleil initialise avec capacite 1 mais body_simulate lui ajoutait N_STEPS points
- Halley simulee deux fois de suite par erreur (copier-coller)
- body_init_point appele deux fois sur le Soleil

Solution

Initialiser le Soleil en premier avec capacite adequate, simuler Halley une seule fois, et verifier systematiquement qu'aucun corps n'est initialise deux fois.

5.4 Satellites initialises avec la mauvaise position

Probleme

init_satellite_orbit lisait le DERNIER point de la trajectoire de la planete parente. Apres system_simulate, la planete avait 87601 points et le dernier point etait sa position apres 5 ans. Le satellite etait donc initialise a cote de la planete apres 5 ans, pas au debut de la simulation.

Solution

Initialiser tous les satellites AVANT system_simulate, quand les planetes n'ont encore qu'un seul point (leur position initiale). L'ordre dans solar_system.c est maintenant : init_planetes, init_satellites, puis system_simulate.

5.5 Debug : valeurs garbage dans s_cur

Probleme

Le debug montrait sat[0] pos = (1.727e-304, ...) — valeurs typiques de memoire non initialisee malloc. Cause : le print de debug etait place AVANT l'initialisation de s_cur dans la boucle.

Solution

Deplacer le print APRES l'initialisation. Lecon : toujours verifier l'ordre des operations dans une boucle avant de conclure a un bug physique.

6. Extensions realisees au-dela du sujet

6.1 Orbites elliptiques avec formule vis-viva

Le sujet utilisait la formule d'orbite circulaire $v_0 = \sqrt{G * M / r}$ pour initialiser toutes les planetes. Notre implementation utilise la formule vis-viva qui est valable pour n'importe quelle excentricite :

```
// Semi-grand axe depuis perihelie et excentricite
double a = perihelion / (1.0 - eccentricity);
// Formule vis-viva : valable a n'importe quel point de l'orbite
double v0 = sqrt(G * M_SUN * (2.0 / perihelion - 1.0 / a));
```

Source : Wikipedia, 'Loi des forces vives', aussi utilisee pour la comete de Halley ($e=0.967$). Cette formule est derivee de la conservation de l'energie mecanique dans un champ gravitationnel central.

Impact visible : Mercure ($e=0.2056$) a une orbite nettement plus elliptique qu'avec l'approximation circulaire. La vitesse au perihelie est 47 900 m/s contre 39 400 m/s a l'aphelie — facteur 1.21 de difference.

6.2 Inclinaisons orbitales en 3D

Le sujet demandait de travailler en 3D mais permettait de laisser $z=0$ pour commencer. Notre implementation inclut les vraies inclinaisons orbitales par rapport a l'ecliptique. La vitesse initiale est tournee dans le plan YZ selon l'angle d'inclinaison :

```
Vector3 vel = {
    0.0,
    v0 * cos(inclination), // composante Y
    v0 * sin(inclination)  // composante Z non nulle
};
```

Mercure avec 7.005 deg est la plus visible. Les valeurs proviennent des NASA Planetary Fact Sheets.

6.3 Comete de Halley

La comete de Halley n'etait pas mentionnee dans le sujet mais est suggeree comme extension possible. Caracteristiques :

- Excentricite $e = 0.967$ (orbite tres allongee)
- Perihelie : $8.766e+10$ m (entre Mercure et Venus)
- Aphelie : $2.667e+12$ m (au-dela de Neptune)
- Inclinaison : 162.26 deg (orbite retrograde — sens inverse des planetes)
- Periode : ~75 ans — simulee sur 10 ans

La vitesse au perihelie calculee par vis-viva est d'environ 54 500 m/s, soit presque le double de la vitesse de la Terre. Source : Wikipedia 'Comete de Halley', NASA JPL Small-Body Database.

6.4 Conservation de l'energie

Le sujet demandait de verifier la conservation de l'energie (section 2.3). Notre implementation calcule a chaque appel :

```
Ec = 0.5 * masse * ||vitesse||^2
Ep = -G * m1 * m2 / ||r1 - r2||
E = Ec + Ep (doit etre constante)
```

La fonction `energy_check_conservation` compare l'énergie initiale et finale et affiche le pourcentage de dérive. Ces résultats sont utilisés pour justifier le choix de la méthode RK2 symplectique.

6.5 Barycentre et mouvement du Soleil

Dans la simulation finale, le Soleil n'est pas fixe — il est soumis à l'attraction de Jupiter et des autres planètes et se déplace autour du barycentre du système solaire. Résultat après 5 ans :

```
Soleil position initiale : (0.000e+00, 0.000e+00, 0.000e+00)
Soleil position finale   : (1.628e+09, 1.931e+09, 9.419e+07)
```

Le déplacement de ~2.5 millions de km est principalement dû à Jupiter. Dans la réalité, le Soleil oscille autour du barycentre du système solaire sur un rayon d'environ 700 000 km (source : NASA, 'The Wobbling Sun').

7. Format d'echange JSON

7.1 Format impose par le sujet

Le sujet imposait un format JSON strict (section 3.1.1). Chaque trajectoire est une liste de points au format `[[x,y,z],[vx,vy,vz],t]` avec les nombres en notation scientifique (%e en C). Nous avons respecte ce format exactement.

7.2 Sous-echantillonnage

Probleme : avec $dt=1800s$ sur 5 ans, chaque corps a 87 601 points. $20 \text{ corps} * 87\,601 \text{ points} * 80 \text{ octets} \approx 140 \text{ Mo}$ — trop lourd pour le navigateur web.

Solution : exporter 1 point toutes les 48 iterations, soit 1 point par jour simulé. Chaque corps a 1 826 points dans le JSON. Taille finale : environ 3 Mo.

La simulation reste calculee a haute resolution ($dt=1800s$) et le sous-echantillonnage ne concerne que l'export. Les calculs physiques ne sont pas affectes.

dt calcul	1800s (30 minutes)
dt export	86400s (1 jour) — sample_every=48
Points par corps (calcul)	87 601
Points par corps (JSON)	1 826
Corps exportes	20 (8 planetes + 11 satellites + Halley)
Taille JSON finale	~3 Mo

8. Sources et references

8.1 Sources demandeées par le sujet

- Sujet ISEN Kepler 2026 — formules equations 1 a 27, methodes Euler et RK2
- L. Garcin — Methode d'Euler et gravitation : <https://lgarcin.github.io/2016-09-26-gravitation/>
- D. Lavabre, V. Pimienta — Cinetique chimique, integration numerique (figure RK2)

8.2 Sources des constantes physiques

- NASA Planetary Fact Sheets — masses, perihelies, inclinaisons, excentricites
- <https://nssdc.gsfc.nasa.gov/planetary/factsheet/>
- Wikipedia — Perihelie (formule vitesse au perihelie utilisee pour initialisation)
- Wikipedia — Comete de Halley ($e=0.967$, periode, inclinaison retrograde)
- Wikipedia — Loi des forces vives (formule vis-viva)
- NASA JPL Small-Body Database — donnees orbitales comete de Halley

8.3 References scientifiques sur les methodes utilisees

- Wisdom, J. & Holman, M. (1991) — 'Symplectic maps for the N-body problem'
- The Astronomical Journal, 102, 1528. Methode multi-echelle pour systemes planetaires.
- Tamayo, D. et al. (2019) — 'REBOUND : an open-source multi-purpose N-body code'
- Journal of Open Source Software. Utilise RK2 symplectique et substeps adaptatifs.
- Leimkuhler, B. & Reich, S. (2004) — 'Simulating Hamiltonian Dynamics'
- Cambridge University Press. Reference sur les integrateurs symplectiques.
- Euler, L. (1768) — 'Institutionum calculi integralis'. Methode d'Euler originale.
- Runge, C. (1895), Kutta, M.W. (1901) — methodes Runge-Kutta.

8.4 Qui utilise les memes methodes

Organisation	Methode	Application
NASA JPL Horizons	Integrateur symplectique + RK adaptatif	Ephemerides officielles des corps du systeme solaire
ESA GAIA mission	Integrateur symplectique d'ordre eleve	Catalogue de 1.8 milliard d'etoiles
REBOUND (Univ. Toronto)	WHFast (RK2 symplectique + substeps)	Simulation systemes exoplanetaires
SpaceX Starship	Integrateur RK4 + corrections temps reel	Guidage et navigation des fusees
Jeux video (Kerbal Space)	Euler asymetrique (Verlet)	Simulation orbital temps reel

A l'oral

Pour repondre a la question 'pourquoi RK2 symplectique ?' : c'est la meme famille de methodes qu'utilise le simulateur REBOUND developpe a l'Universite de Toronto et utilise dans des publications scientifiques de mecanique celeste. Notre implementation avec substeps adaptatifs suit le meme principe que l'integrateur WHFast (Wisdom-Holman Fast) de REBOUND.

9. Bilan et ce qu'on aurait pu ameliorer

9.1 Ce qui a ete fait

Fonctionnalite	Statut
Euler simple (minimum demande)	Termine
Export JSON format impose	Termine
Euler asymetrique (extension sujet)	Termine
RK2 (extension sujet)	Termine
Conservation de l'energie	Termine
RK2 symplectique (notre extension)	Termine
8 planetes + 11 satellites	Termine
Axe Z avec inclinaisons reelles	Termine
Orbites elliptiques vis-viva	Termine
Comete de Halley	Termine
Integration multi-echelle substeps	Termine
Barycentre + mouvement du Soleil	Termine
Sous-echantillonnage export JSON	Termine

9.2 Limites connues

- Les attrapeurs utilises pendant les substeps sont les positions du debut du pas global, pas leurs positions interpolees. Sur 1800s, Mars bouge de 43 200 km — acceptable mais pas parfait.
- Pas de precession des perihelies (effet relativiste de Mercure). Impossible sans relativite generale.
- Halley simulee separement du systeme solaire — elle ne recoit pas l'attraction de Jupiter qui la perturbe dans la realite.
- Les satellites sont initialises sur des orbites circulaires, pas leurs vraies orbites elliptiques.
- L'orientation des ellipses (argument du perihelie) n'est pas implementee — toutes les planetes partent du meme cote.

9.3 Ce qu'on aurait fait avec plus de temps

- Integrateur IAS15 d'ordre 15 (REBOUND) pour une precision maximale
- Donnees initiales reelles depuis les ephemerides NASA Horizons
- Vaisseau spatial avec catapulte gravitationnelle autour de Jupiter
- Prediction des eclipses et des alignements planetaires